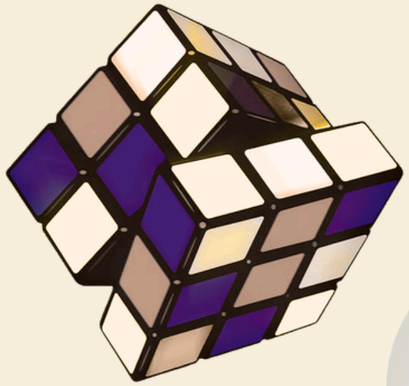
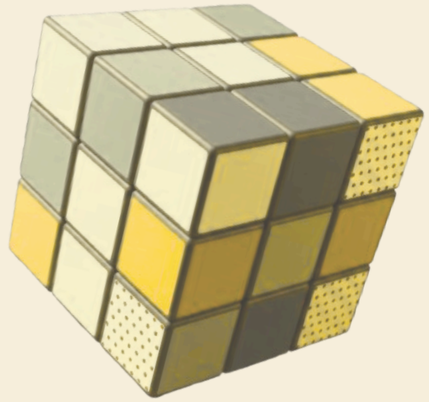




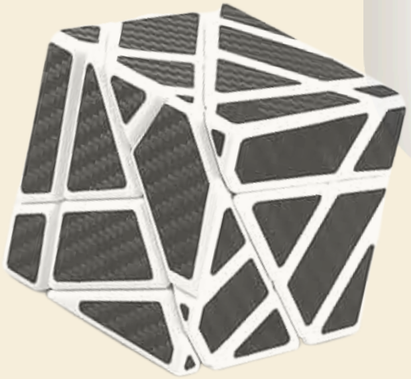
Plastic_box_solver

SWE-4404 SPL II

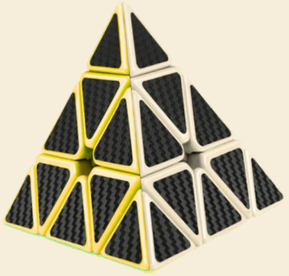
Final Presentation

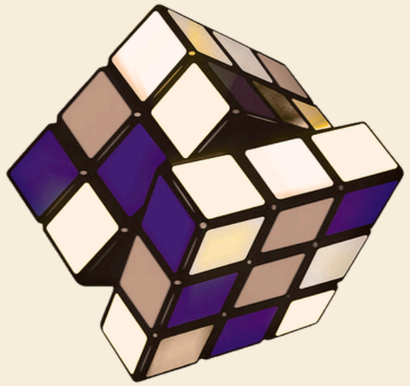
TEAM 06

S.M. Tahsinuzzaman Emon	230042104
S.M. Samiul Hossain	230042110
MD. Muntahi Hasan Akhiar	230042118

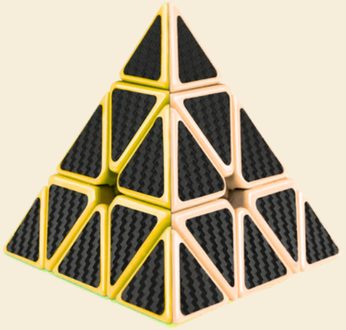






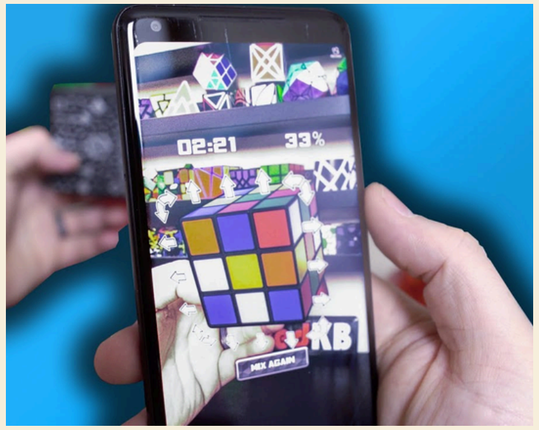


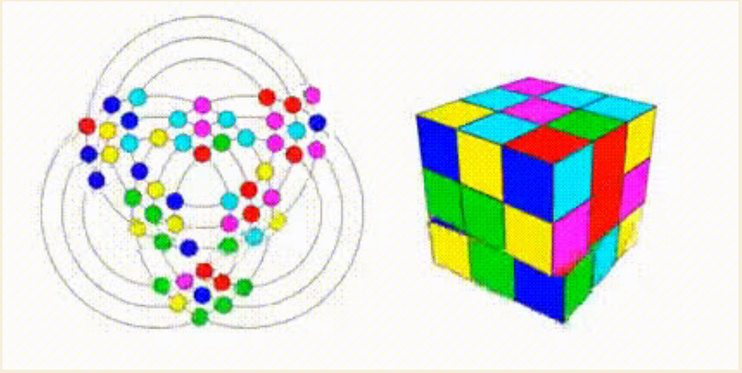
INTRODUCTION



Point. Detect. Solve.

A web-based 3D simulation model of Rubik's Cube that scans your cube, lets you fix it and guides you to solve it step by step all in your browser and keep track of your stats.





R U R' U' R' F R2 U' R' U' R U R' F'





FEATURES



Input

- Take Input from Live Camera Feed
- Manual Input for Correction in case of error on scan

Cube Simulation

- Full 3D Rotatable Cube
- Shows Preview of the whole cube on six (6) 2D Plane
- Learn/ Test your skills



FEATURES



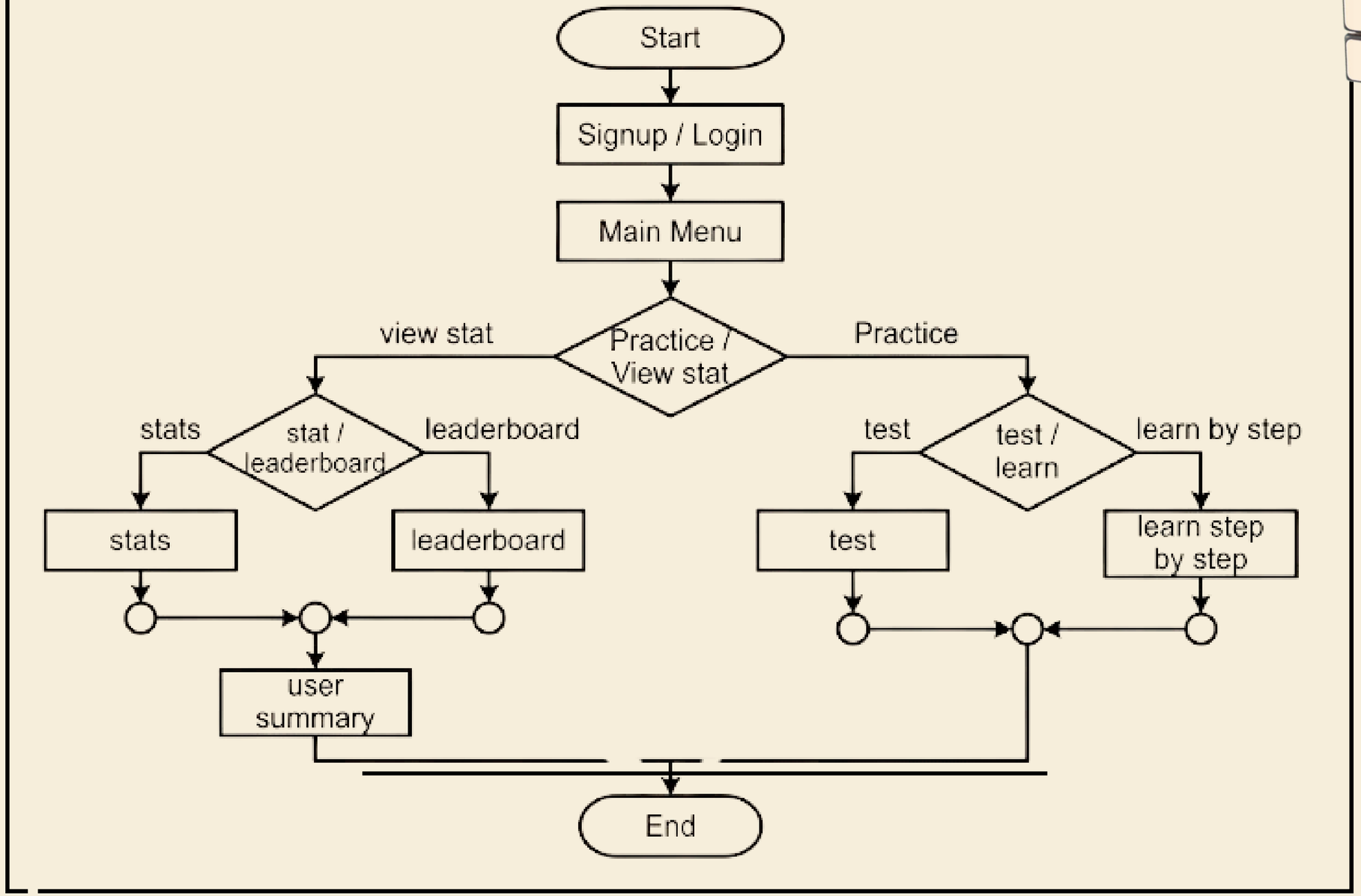
Modes

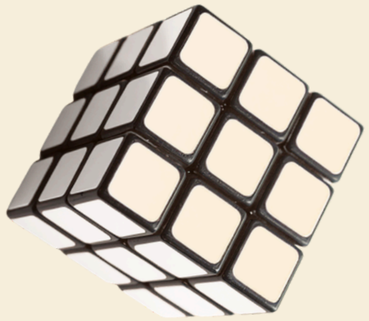
- Interactive Solve
  - Learn (Guides the user on how to solve)
  - Test (Lets user test with the scanned cube)
- Auto Solve - System Shows the most efficient Solution

Stats

- Current Solve Stat
- Compare Stat with personal Best
- Total Solve and Efficiency with History
- Global Leaderboard

FLOWCHART


Start
Signup / Login
Main Menu
Practice / View stat
view stat
Practice
stat / leaderboard
stats
leaderboard
test / learn
test
learn by step
stats
leaderboard
test
learn step by step
user summary
End



TOOLS & TECHNOLOGY

FRAMEWORKS

INPUT & PROCESSING

OpenCV

3D RENDERING

API & TESTING

VERSION CONTROL

git

DATABASE



PROJECT STRUCTURE



```
plastic-box-solver/
├── docker-compose.yml
└── code/
    ├── backend/ (Python / FastAPI Application)
    │   ├── app/ (Application core)
    │   │   ├── cube/ (Pure Domain: Cube engine & algorithms)
    │   │   ├── vision/ (Computer Vision subsystem)
    │   │   ├── routers/ (HTTP Controller layer / API endpoints)
    │   │   ├── db/ (Persistence layer)
    │   │   └── utils/ (Cross-cutting utilities)
    │   └── tests/ (Backend automated tests)
    └── frontend/ (React 18 + Three.js Single Page Application)
        ├── src/ (Frontend source)
        │   ├── Layout/ (Persistent UI shell & navigation)
        │   ├── Pages/ (Full-page views & workspace)
        │   ├── Components/ (Modular reusable UI widgets)
        │   ├── Three/ (Three.js 3D graphics & rendering)
        │   ├── cube/ (Mirrored client-side cube engine)
        │   ├── store/ (Zustand global state management)
        │   └── lib/ (API & client utilities)
        └── tests/ (Client-side tests)
```



COMMUNICATION BETWEEN COMPONENTS



request DTO
(frotnend to backend)

```python
# app/schemas.py

from Pydantic import schemas.Pydantic

class SolveCreate(BaseModel):
    solve_time: float = Field(..., ge=0)
    move_count: int = Field(..., ge=1)
    optimal_moves: int = Field(..., ge=0)
    method: str = "CFOP"
    scramble: Optional[str] = None
    solution: Optional[str] = None
```

response DTO
(backend to frontend)

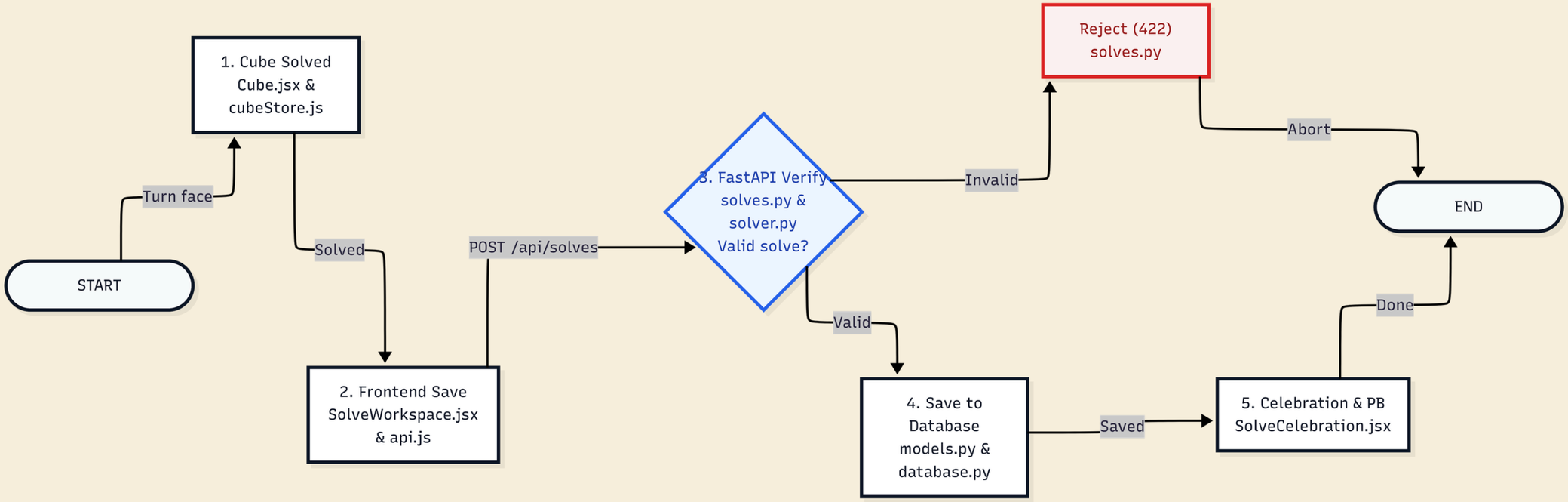
```python
#app/schemas.py
class SolveOut(BaseModel):
    id: int
    solve_time: float
    move_count: int
    optimal_moves: int
    efficiency: float
    method: str
    created_at: datetime
    scramble: Optional[str] = None
    solution: Optional[str] = None

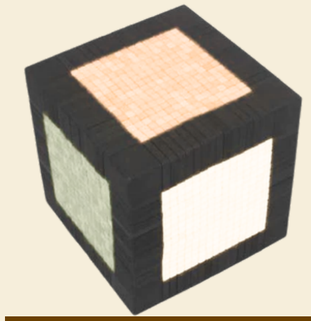
    class Config:
        from_attributes = True
```



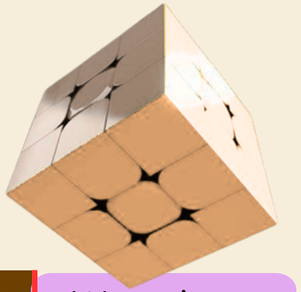
END TO END EXAMPLE





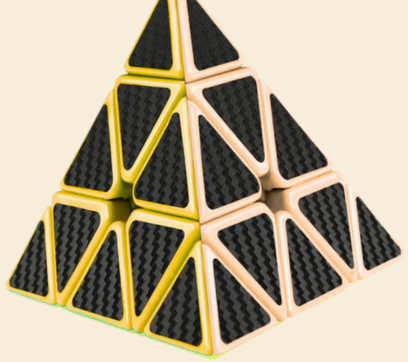


TIMELINE



We are here

Task List	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12	Week 13	Week 14
Team Formation & Idea generation												
Frontend & UI												
3D Cube												
Opencv Pipeline												
Input processing												
Solving Algorithm												
Backend & Api												
Integrating & Merging												
Debugging & testing												





Thank You

